

dev tra Git編

チーム制作のためのGit入門（全14回）

【1-01】Gitってなに？なんで必要？【1章：Git導入編】

1. クリア条件：Gitが何をしてくれる道具なのか、人に説明できるようになる

2. ここからは別巻、チーム制作のためのGit編です。

今回は手を動かしません。まず「なんでこんなもの覚えるの？」をハッキリさせておきましょう。ここが分かっていないと、この先の操作がぜんぶ意味不明な呪文になってしまうので。

3. まず、こんな経験ありませんか？

```
キャラ絵_最終.psd  
キャラ絵_最終2.psd  
キャラ絵_最終_修正版.psd  
キャラ絵_ほんとの最終.psd  
キャラ絵_ほんとの最終_これで決定.psd
```

あるあるですね。これ、実は自分ひとりで手動バージョン管理をやっている状態です。

そして大体、3日後には「あれ、どれが最新？」「あの時の色に戻したいけど、どのファイルだっけ？」となります。

4. Gitは、この「名前で管理する地獄」から解放してくれる道具です。

ファイルは常に「キャラ絵.psd」1つだけ。そのかわり、変更するたびに**セーブポイント**を刻んでいきます。

「いつ」「誰が」「どこを」変えたのが全部記録されるので、いつでも好きな時点に戻れます。

用語：Git（ギット）

ファイルの変更履歴を記録・管理してくれる道具。ゲームで言うところの**セーブ&ロード機能**。しかも上書きセーブではなく、全部のセーブデータが残る親切設計。

5. でも、Gitの本当の価値はチーム制作で出ます。

たとえば、あなたが敵キャラの絵を描いて、プログラマーがスクリプトを書いているとします。GitなしでUnityプロジェクトを共有すると、どうなるか？

- Aさん「プロジェクトフォルダzipで送るね」
- Bさん「ありがとう。こっちも直したから送り返すね」
- Aさん「え、その間にこっちも直しちゃった。どっちが正しいの？」
- 二人「……」

よくある事故です。zipを送り合う運用は、必ずどこかで誰かの作業が消えます。

6. Gitを使うと、こうなります。

- 全員が同じ「倉庫」を見る
- 各自が自分の作業をして、終わったら倉庫に納品する
- Gitが「Aさんは絵を、Bさんはスクリプトを変えたんだな」と判断して、**自動で合体**してくれる

この「自動で合体」がGitの真骨頂です。違うファイルに触っている限り、二人の作業は勝手に混ざって1つになります。

用語：マージ

別々に進めた作業を合体させること。Gitがほぼ自動でやってくれる、いちばんありがたい機能。

7. まとめて、Gitはこの3つをやってくれます。

できること	たとえるなら
変更履歴を全部残す	いつでも戻れるタイムマシン
複数人の作業を合体する	全員の作業が合流する共有セーブデータ
誰が何をしたか記録する	作業ログ付きの制作日誌

8. 「デザイナーだからプログラマーに任せればいいのでは？」と思った人、鋭い。でも残念、無理です。

Unityプロジェクトでは、絵を1枚追加するだけでもプロジェクトのファイルが変わります。絵もアニメも音も、全部Gitの管理下に入ります。

つまりチーム制作をする以上、**職種に関係なく全員が使うのがGit**です。逆に言えば、これさえ覚えれば「素材を渡してからの反映待ち」がなくなり、自分のペースで作業を進められるようになります。

9. 最後に、心構えをひとつ。

Gitは最初、絶対に意味不明です。カタカナ用語（コミット、プッシュ、ブランチ、マージ...）が大量に出てきて、たぶん3回くらい「無理」と思います。

でも安心して下さい。この先で覚える操作は、実質**4つ**だけです。

- **コミット**（セーブする）
- **プッシュ**（倉庫に送る）
- **プル**（倉庫から取ってくる）
- **ブランチ**（作業場所を分ける）

残りは全部この応用です。しかも今回は黒い画面（コマンド）は一切使いません。ボタンをポチポチするだけのアプリでやります。

10. Gitが「タイムマシン付きの共有倉庫」だとイメージできたら、今回は完成です！

次回はその倉庫を借りにいきます。

11. ↓ --- **質問コーナー** --- ↓

【1-02】GitHubアカウントを作ろう【1章：Git導入編】

1. クリア条件：GitHubのアカウントを作って、ログインできる状態にする

2. 前回、Gitは「タイムマシン付きの共有倉庫」だと説明しました。

では、その倉庫はどこにあるのか？——それが今回のGitHub（ギットハブ）です。

用語：GitとGitHubの違い

よく混同されるので最初にハッキリさせておきます。

- **Git** = 道具（あなたのPCに入れる、履歴を管理するソフト）
- **GitHub** = 場所（ネット上にある、みんなでファイルを置いておく倉庫サービス）

Photoshopが「Git」で、Google Driveが「GitHub」みたいな関係だと思ってください。片方だけでは成立しません。

3. ちなみにGitHubは黒い猫（正確にはタコと猫を合体させた「Octocat」というマスコット）が目印のサイトです。世界中のプログラマーが使っていて、無料で使えます。

4. では作りましょう。ブラウザで下のURLを開いてください。

<https://github.com/signup>



アクセスは一時的に制限されています

これが表示されたのは、ブラウザによる不審な挙動が検知されたためです。

以下のような理由が考えられます：

- ページ遷移やクリックが異常に速い
- お使いのコンピュータ上でJavascriptの機能がブロックされている
- 同じネットワーク(IP 153.240.242.132)上にロボットがある

サイトへのアクセスに問題がありますか？ [サポートに連絡する](#)

ID: 19302724-11ca-3d9b-3226-95e8bda1eba4

5. 入力するのは3つだけです。

- **Email**：普段使っているメールアドレス
- **Password**：15文字以上、または8文字以上 + 数字と小文字を含む
- **Username**：あなたのID。**チームメンバーに見える名前**なので、あだ名など呼ばれ慣れた名前がおすすめ

※Usernameはあとから変更できますが、URLが変わって面倒なので、最初に決め打ちしましょう。半角英数とハイフンのみ使えます。

6. 「Country/Region」は Japan を選択。メール配信の希望は好みでどうぞ（チェックしなくてOK）。

7. 「Create account」を押すと、メールに6桁の確認コードが届きます。入力すれば完了です。

※届かない人は迷惑メールフォルダを確認してください。

8. その後、「What do you want to do first?」みたいなアンケート画面が出てきますが、全部スキップして大丈夫です。右下の「Skip personalization」や「Continue for free」を選びましょう。

※有料プランを勧められますが、**Freeで十分**です。

9. ログインした状態で、自分のトップページが表示されたら成功です！

まだ何もない真っさらな状態ですが、これであなたの倉庫を借りる権利が手に入りました。

10. 最後に、**Username**を**チームのリーダー**に伝えておきましょう。

リーダーがあなたを「このプロジェクトの共同編集者」として招待するのに必要です。招待されないと、倉庫に納品する権限がもらえません。

※Discordなどで「GitHubのIDは〇〇です」と一言送っておけばOK。

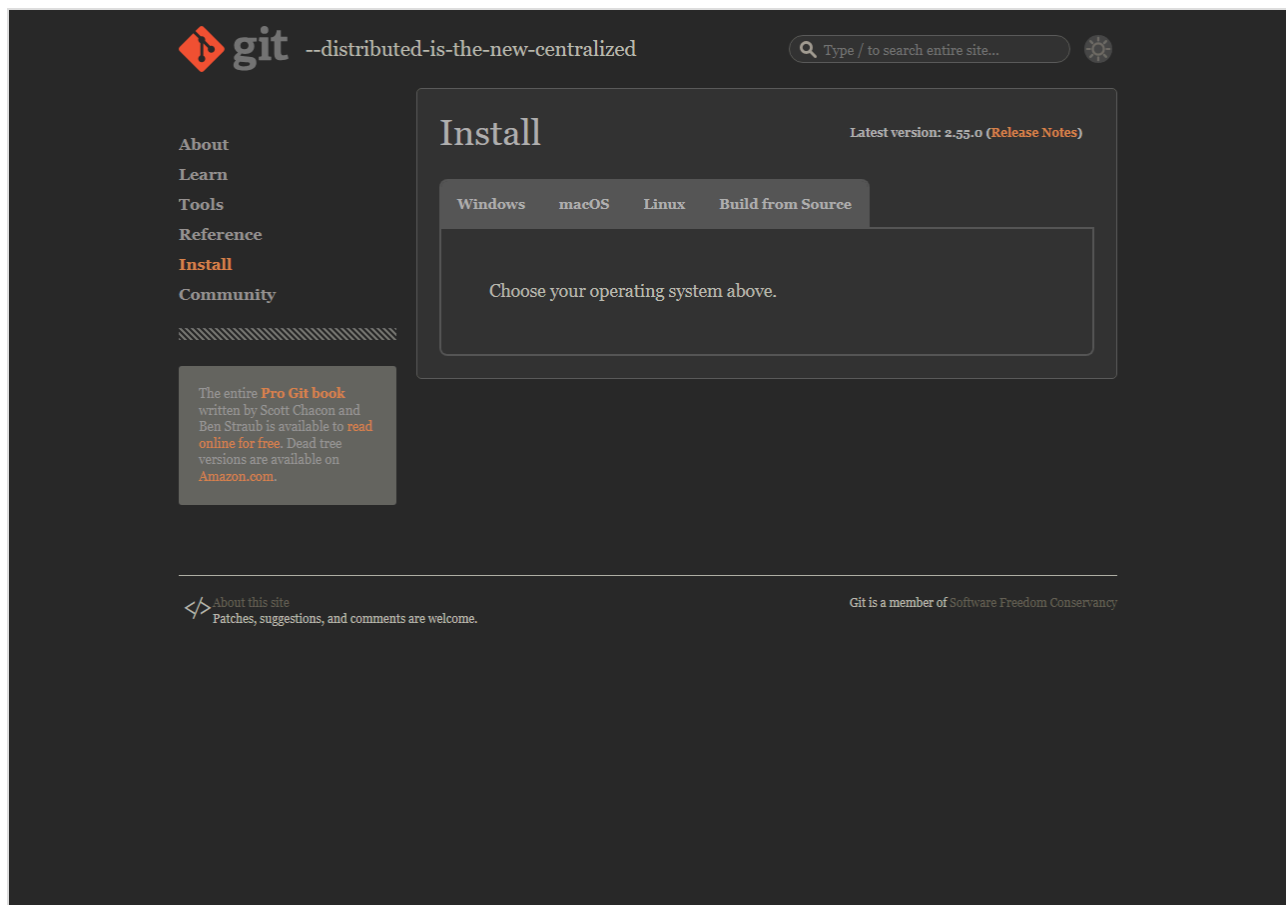
11. GitHubにログインできて、IDをチームに伝えたら、今回は完成です！

12. ↓ --- **質問コーナー** --- ↓

【1-03】Gitをインストールしよう【1章：Git導入編】

1. クリア条件：自分のPCにGit本体を入れる
2. 前回は「倉庫」を借りました。今回は「道具」であるGit本体をPCに入れます。
これはエンジンみたいなものです。この後で入れるアプリ（Fork）は、いわばハンドルとメーター。エンジンがないと動きません。
3. まずは、すでに入っていないか確認しましょう。実はUnityやVisual Studioを入れたときに、一緒に入っていることがあります。
Windowsの人：スタートメニューで「Git Bash」と検索してみてください。出てきたら、もう入っています。→ 手順8へ
Macの人：後述の手順7を見てください。
4. 入っていなかった人は、公式サイトからダウンロードします。

<https://git-scm.com/downloads>



※黒背景に赤いロゴの、ちょっと無骨なサイトです。合っています。

5. 【Windowsの人】「Windows」を選び、「Click here to download」から最新版をダウンロード。
ダウンロードした .exe をダブルクリックして進めます。
6. 【Windowsの人】インストーラーの設定項目がやたら多くて心が折れそうになりますが、**全部そのまま「Next」で大丈夫**です。
デフォルト設定がちゃんと考えられているので、いじる必要はありません。

最後に「Install」→「Finish」で完了。

※途中で「デフォルトのエディタを選べ」という画面が出ます。ここもそのまま（Vim）で問題ありません。今回は使いませんので。

7. 【Macの人】Macは少し楽です。「ターミナル」アプリを開いて、`git --version` と打ってEnter。

入っていないければ「開発者ツールをインストールしますか？」というダイアログが出るので、「インストール」を押すだけで入ります。

※Homebrewを使っている人は `brew install git` でもOKです。

8. 入ったかどうか確認しましょう。

Windowsは「Git Bash」、Macは「ターミナル」を開いて、こう打ちます。

```
git --version
```

`git version 2.55.0` のようにバージョン番号が返ってきたら成功です！

※ここだけ黒い画面を使いますが、確認のためだけです。この先はもう黒い画面は出てきません。安心してください。

9. 最後に、あなたの「署名」を設定します。

Gitは変更履歴に「誰がやったか」を記録するので、名前とメールアドレスを教えてあげる必要があります。これをやらないと、後でコミットするときに怒られます。

同じ黒い画面で、下の2行をそれぞれ実行してください（`"`の中は自分のものに書き換え）。

```
git config --global user.name "あなたの名前"
```

```
git config --global user.email "GitHubに登録したメールアドレス"
```

※名前は日本語でも大丈夫ですが、ローマ字やGitHubのIDにしておくとも後々見やすいです。

※メールアドレスは1-02でGitHubに登録したものにしてください。ここが違うと、GitHub上で「誰の作業か」が紐づきません。

10. `git --version` が表示されて、署名の設定ができれば、今回は完成です！

次回は、これをマウス操作で使えるようにするアプリを入れます。

11. ↓ --- 質問コーナー --- ↓

【1-04】Forkを入れよう（黒い画面とお別れ）【1章：Git導入編】

1. クリア条件：GitのGUIアプリ「Fork」を入れて、GitHubアカウントと連携する
2. 前回入れたGit本体は、実は**エンジン単体**なので、そのままと黒い画面にコマンドを打って操作することになります。
`git commit -m "..."` とか `git push origin feature/hoge` とか。.....正直、これを毎日打ちたい人はいませんよね。
3. そこで、Gitをマウス操作で使えるようにするアプリを入れます。これをGUIクライアントと呼びます。

用語：GUIクライアント

黒い画面（コマンド）の代わりに、ボタンやアイコンでGitを操作できるアプリ。中身は同じGitを動かしているだけなので、できることは全く同じです。Photoshopで言えば、コマンドが「スクリプト」、GUIが「ツールパネル」の関係です。

4. 今回使うのは **Fork** というアプリです。理由は3つ。
 - 変更履歴が**枝分かれの図**で見える（これが直感的に分かりやすい）
 - どのファイルが変わったか、画像もサムネイルで確認できる
 - 動作が軽い

※Mac版・Windows版の両方があり、**画面がほぼ同じ**なのも採用理由です。チームでOSが混ざっていても、同じ説明が通じます。

5. では入れましょう。下のURLを開いてください。

<https://git-fork.com/>



Fork is getting better and better day after day and we are

自分のOSのボタン（Download for Windows / Download for Mac）を押してダウンロード、インストールします。

※Forkは「無期限で試用できる有料アプリ」です。起動のたびに購入案内が出ますが、機能制限なく使い続けられます。気に入ったら買ってあげてください（買い切り \$59.99）。

6. 初回起動すると、名前とメールアドレスを聞かれます。1-03で設定したものと同じものを入れましょう。（すでにGitに設定済みなら、自動で入っているはずです）

📷 スクショ：Forkの初回設定画面

7. 次に、GitHubアカウントと連携します。これをやっておくと、毎回パスワードを聞かれずに済みます。

- Windows：File > Preferences
- Mac：Fork > Preferences（またはSettings）

を開いて、「Integration」（または「Accounts」）タブへ。

8. 「GitHub」の欄で「Add account」または「Sign in」を押すと、ブラウザが開いてGitHubの認証画面が出ます。「Authorize fork」（緑のボタン）を押せば連携完了です。

📷 スクショ：ForkのGitHub連携設定

9. 連携できると、Forkに自分のGitHubアカウント名が表示されます。ここまでで環境構築は完了です！

10. 【補足】チームでVSCodeを使っている人へ

Visual Studio Code にも、実はGit機能が最初から入っています（左端の枝分かれアイコン）。

コードを書きながらそのままコミットできるので便利ですが、履歴の枝分かれが見つらいのが難点です。

おすすめは併用です。「日々のコミットはVSCode、ブランチ操作や履歴確認はFork」という使い分けをしている人が多いです。もちろん、全部Forkでも構いません。

11. Forkが起動して、GitHubと連携できたら、今回は完成です！

次回からいよいよ実際のプロジェクトを触ります。

12. ↓ --- 質問コーナー --- ↓

【1-05】Unity用の.gitignoreを知ろう【1章：Git導入編】

1. クリア条件：.gitignoreが何のためにあるか説明できて、Unity用のものを用意できる
2. 今回もちっと座学寄りですが、**ここを飛ばすとチーム制作が確実に事故ります**。5分だけお付き合いください。
3. まず前提として、Unityのプロジェクトフォルダを開いてみてください。こんなフォルダが並んでいます。

Assets/	← 自分で作った素材・スクリプト
ProjectSettings/	← プロジェクトの設定
Packages/	← 使っている機能のリスト
Library/	← ★
Temp/	← ★
Logs/	← ★
obj/	← ★

4. このうち★の付いたフォルダ、実は**Unityが勝手に作る一時ファイル**です。特にLibraryは、あなたが入れた画像や音を、Unityが内部用に変換したデータの置き場。数GBになることもあります。
5. ここで問題です。このLibraryフォルダ、チームで共有すべきでしょうか？
答えは**NO**です。理由は3つ。

- **重すぎる**：数GBを毎回アップロード・ダウンロードすることになる
- **意味がない**：Assetsさえあれば、Unityが各自のPCで自動生成してくれる
- **確実に喧嘩する**：全員のPCで中身が微妙に違うので、毎回「変更あり」と判定されて競合しまくる

たとえるなら、**レシピと作り置き**の関係です。共有すべきなのはレシピ（Assets）だけ。作り置き（Library）は、各自のキッチンで同じレシピから作れるので、送り合う必要がありません。むしろ冷蔵庫を圧迫します。

6. そこで登場するのが .gitignore（ギット・イグノア）です。

用語：.gitignore

「このファイルたちはGitで管理しなくていいよ」というリストを書いた設定ファイル。名前の通り ignore（無視）するものを指定します。ファイル名がドットから始まるので、フォルダ内で先頭に表示されます。

7. Unity用の.gitignoreは、ありがたいことに**GitHubが公式のテンプレートを用意してくれています**。自分で書く必要はありません。
次回リポジトリを作るときに、選択肢から「Unity」を選ぶだけで自動生成されます。今回は中身を眺めるだけでOKです。

```
# Unityが自動生成するフォルダ（共有しない）
[Ll]ibrary/
[Tt]emp/
[Oo]bj/
```

```
[Bb]uild/  
[Ll]ogs/  
  
# ビルドの成果物（共有しない）  
*.apk  
*.unitypackage  
  
# OSが勝手に作るゴミファイル  
.DS_Store          ← Macが作る隠しファイル  
Thumbs.db          ← Windowsが作るサムネイルキャッシュ
```

8. 【超重要】Unity特有の注意点：.metaファイルは必ず一緒にコミットする

Assetsフォルダを見ると、画像1枚につき「Player.png」と「Player.png.meta」の2つがあるはずです。

この.metaファイルには、**その素材の設定**（Pixels Per Unit、Sprite Modeなど）と、**Unity内部での通し番号（GUID）**が書かれています。

もし.metaを共有し忘れると、こうなります。

- あなたのPCでは、Playerオブジェクトに絵がちゃんと付いている
- 相手のPCでは、**絵が外れて白い四角（Missing）**になる

Unityあるあるの事故です。「なんか相手の環境だけ絵が消える」の犯人はだいたいこれ。

Forkを使っていれば、.metaも普通に表示されるので、**変更されたファイルは全部まとめてコミットすると覚えておけば大丈夫**です。

9. 【もうひとつ重要】シーンとプレハブは同時に触らない

.gitignoreの話とセットで覚えてほしいのがこれです。

Gitは、テキストファイルなら「Aさんは3行目、Bさんは50行目を変えたな」と判断して自動合体できます。

でもUnityのシーンファイル（.unity）やプレハブ（.prefab）は、中身が複雑なテキストなので、**同じシーンを2人が同時に編集すると、ほぼ確実に合体に失敗します（コンフリクト）**。

なので、チームでの鉄則はこれです。

- **同じシーン・同じプレハブを、同時に2人で触らない**
- 誰がどのシーンを触るか、作業前にDiscordで宣言する
- 大きなシーンは、機能ごとにプレハブに分けて分担する

※これは1-14の作業マニュアルにも再掲します。忘れてもそっちを見ればOK。

10. 「Libraryは共有しない」「.metaは一緒にコミットする」「同じシーンを同時に触らない」の3つが頭に入ったら、今回は完成です！

11. ↓ --- 質問コーナー --- ↓

【1-06】リポジトリを作ろう【1章：Git導入編】

1. クリア条件：GitHubにチーム用のリポジトリを作り、メンバーを招待する
2. いよいよ倉庫を1つ借ります。この倉庫のことを、Gitの世界ではリポジトリと呼びます。

用語：リポジトリ（repository）

直訳すると「貯蔵庫」。プロジェクト1つにつき倉庫1つ、と思ってください。中にはファイルだけでなく、これまでの全変更履歴も一緒に保管されています。

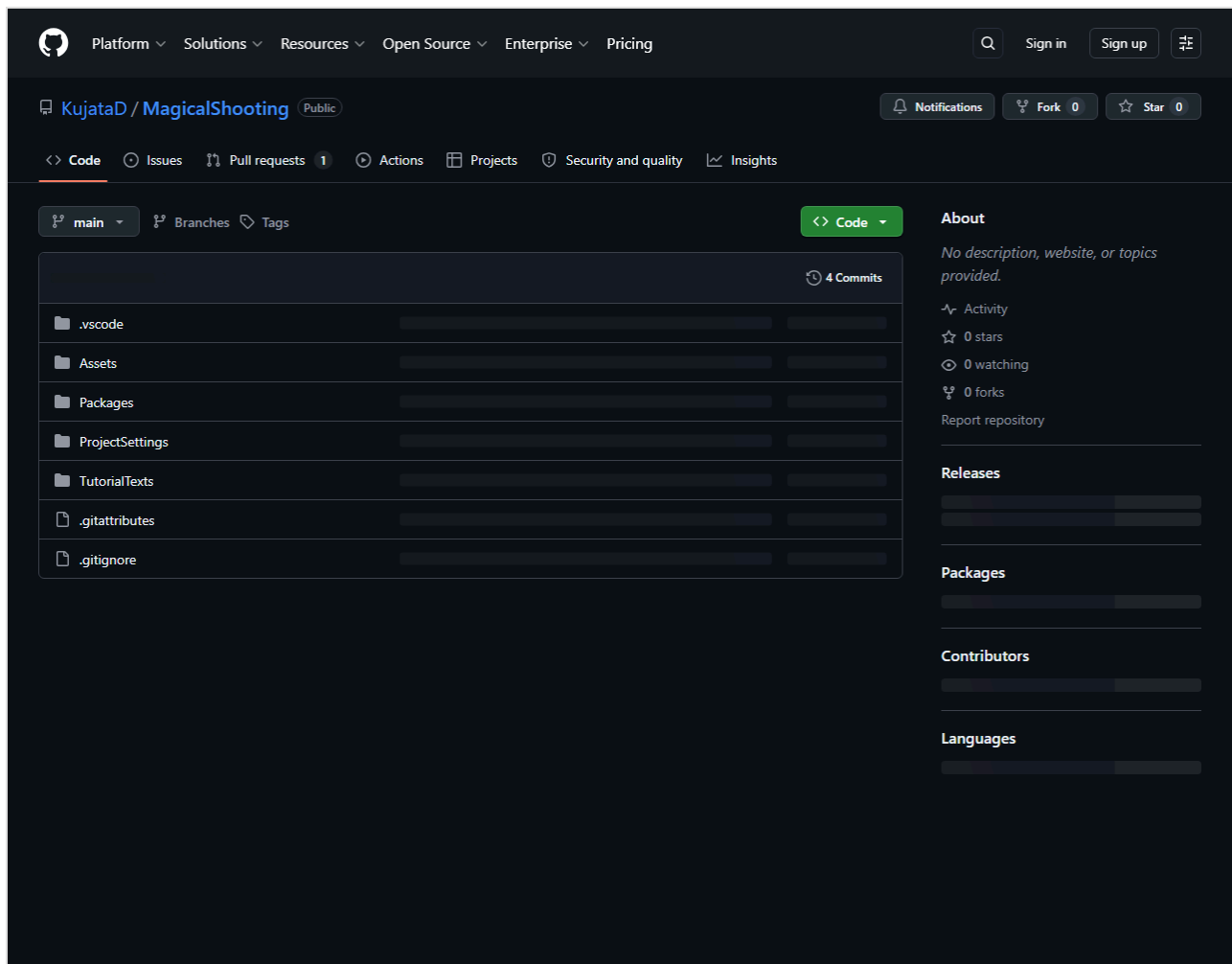
3. ※この作業はチームで1人（リーダー）だけがやればOKです。他のメンバーは眺めるだけで大丈夫。自分がリーダーでない人は、次回の「クローン」から参加になります。
4. GitHubにログインした状態で、右上の「+」→「New repository」を選びましょう。
5. 設定項目を埋めていきます。
 - **Repository name**：プロジェクト名（例：MagicalShooting）。半角英数とハイフン推奨
 - **Description**：どんなゲームか一言（任意）
 - **Public / Private**：
 - Public = 世界中の誰でも見られる
 - Private = 招待した人だけ見られる

チーム制作の練習ならどちらでもOK。素材の権利が気になるならPrivateが安全です

6. 下にスクロールして、ここが今回いちばん大事なところですよ。
 - **Add a README file**：チェックを入れる（プロジェクトの説明書。あとで書けます）
 - **Add .gitignore**：ドロップダウンから **Unity** を選ぶ ← 前回勉強したやつ！
 - **Choose a license**：チーム内制作なら「None」でOK

.gitignoreで「Unity」を選び忘れると、Libraryフォルダが丸ごと共有される地獄が始まります。ここだけは絶対に確認してください。

7. 「Create repository」を押すと、あなたの倉庫が完成します！



※このスクショは、すでに中身が入った状態のものです。作りたてはREADME.mdと.gitignoreだけが入っています。

8. 次に、チームメンバーを招待します。これをやらないと、メンバーは中身を見られても**納品（プッシュ）ができません**。

- リポジトリの上部メニューから「Settings」を開く
- 左メニューの「Collaborators」（共同編集者）を選ぶ
- 「Add people」を押して、メンバーのGitHub IDを入力

1-02でメンバーに「IDを教えてね」と言っておいたのは、このためでした。

9. 招待されたメンバーには、GitHubからメールが届きます。中の「Accept invitation」を押してもらえば、参加完了です。

※招待は7日で期限切れになるので、メンバーには「招待送ったから見てね」と一声かけましょう。

10. 最後に、**リポジトリのURLをチーム全員に共有**しましょう。次回、全員がこのURLを使います。

`https://github.com/ユーザー名/リポジトリ名`

11. リポジトリができて、メンバーが参加できたら、今回は完成です！

次回は、この倉庫の中身を自分のPCに持ってきます。

12. ↓ --- 質問コーナー --- ↓

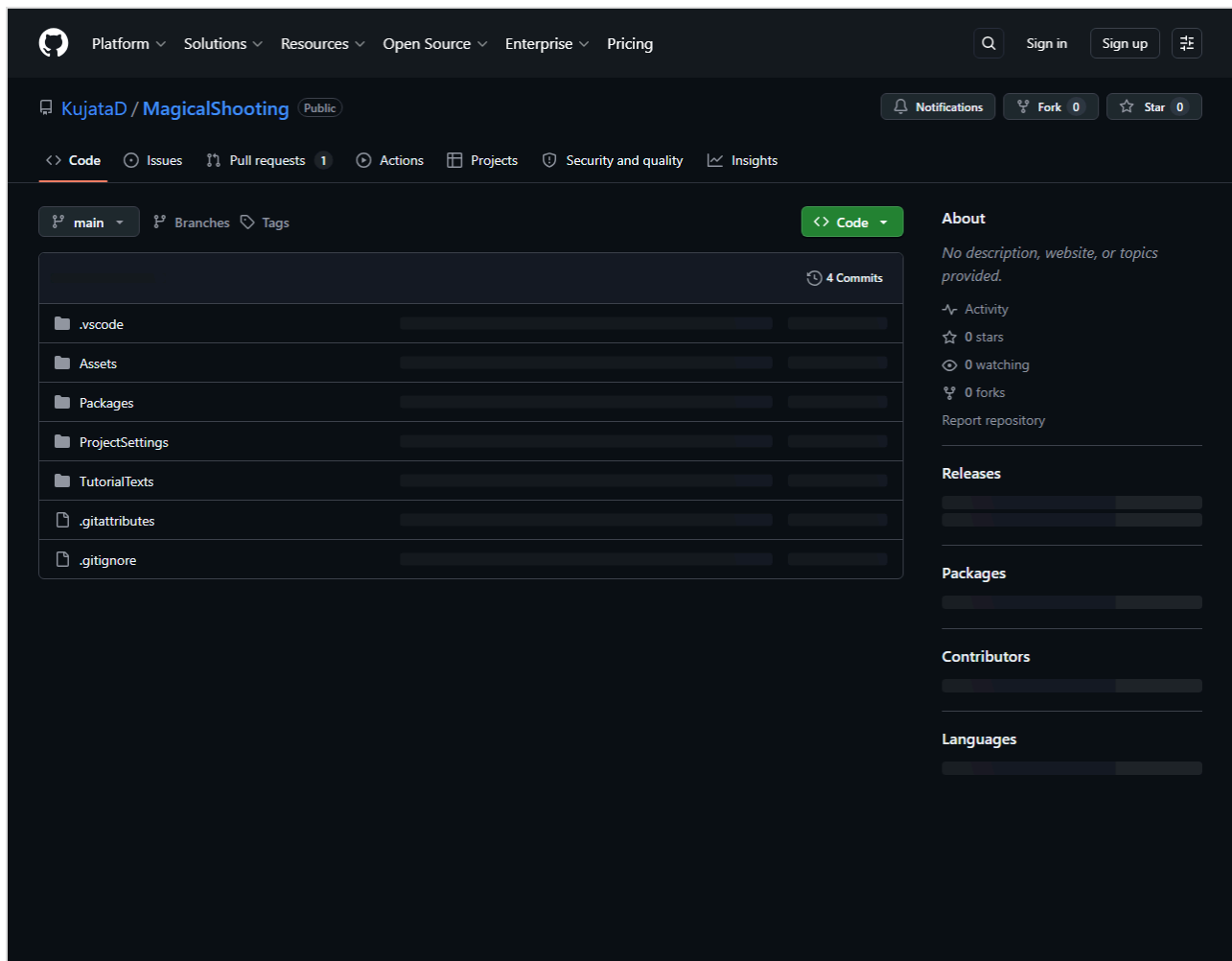
【1-07】クローンしよう（手元に持ってくる）【1章：Git導入編】

1. クリア条件：GitHubのリポジトリを自分のPCにコピーして、Unityで開ける状態にする
2. 倉庫はできましたが、まだネットの向こう側にあります。作業するには手元に持ってこないといけません。
この「持ってくる」操作をクローンと呼びます。

用語：クローン（clone）

リポジトリを丸ごと自分のPCにコピーすること。**ダウンロードとの違いは、履歴も一緒に付いてくること**。単にファイルをもらうのではなく、倉庫ごと複製して、GitHubと繋がった状態にします。

3. ※ここからは**チーム全員がやる作業**です。リーダーも含めて全員、自分のPCに持ってきてましょう。
4. まず、GitHubのリポジトリページを開いて、緑色の「Code」ボタンを押します。
出てきたURL（ <https://github.com/...> で終わるやつ）の右にあるコピーアイコンを押して、URLをコピーします。



5. 次にForkを開いて、File > Clone を選びます。

📷 スクショ：ForkのCloneダイアログ

6. ダイアログに入力するのは2つ。

- **Repository URL** : さっきコピーしたURLを貼り付け
- **Parent Folder** : どこに置くか (例 : `C:\Users\自分の名前\source\repos`)

※「Parent Folder」に指定したフォルダの中に、リポジトリ名のフォルダが自動で作られます。あらかじめ空フォルダを作っておく必要はありません。

7. 【重要】置き場所の注意

ここで指定するフォルダは、次の条件を守ってください。守らないと後で泣きます。

- パスに日本語やスペースを含めない (`C:\Users\山田\デスクトップ` などはNG)
- **OneDriveやGoogle Driveの同期フォルダの中に置かない** ← これ超重要！

後者が特に事故ります。GitとOneDriveが同時に同じファイルをいじろうとして、両方が壊れます。バックアップはGitHubがやってくれるので、クラウド同期は不要です。おすすめは `C:\Users\自分の名前\source\repos` あたり。

8. 「Clone」ボタンを押すと、ダウンロードが始まります。プロジェクトの大きさによっては数分かかります。

9. 完了したら、Forkの画面にファイル一覧と履歴が表示されます。これがあなたの作業場所です。

 スクショ : クローン直後のFork画面

10. では、UnityHubからこのプロジェクトを開いてみましょう。

- UnityHubの「Add」→「Add project from disk」
- さっきクローンしたフォルダを選択

初回はLibraryフォルダを一から生成するので、5〜15分ほどかかります。これが1-05で話した「作り置きを各自のキッチンで作る」作業です。お茶でも飲んで待ちましょう。

11. Unityでプロジェクトが開けたら、今回は完成です！

ここまでで準備完了。次回からいよいよ、作業してGitに記録していきます。

12. ↓ --- 質問コーナー --- ↓

【1-08】コミットしよう（セーブポイントを刻む）【1章：Git導入編】

1. クリア条件：ファイルを変更して、コミットを1回作る
2. いよいよGitの基本動作、コミットです。これはゲームで言う**セーブ**にあたります。

用語：コミット（commit）

その時点のファイルの状態を、履歴として記録すること。上書きセーブではなく、セーブデータが毎回増えていくタイプのセーブ。だから何度でも過去に戻れます。

3. まずは何か変更を作りましょう。何でもいいのですが、練習ならUnityで簡単な変更を1つしてみてください。
（例：シーンにテキストを1つ追加して保存、画像を1枚Assetsに追加、など）
※必ず**Unity側でCtrl+Sして保存**してから、Forkに戻ってください。保存していない変更はGitからは見えません。
4. Forkを開くと、左メニューの一番上に「Local Changes」という項目があり、数字が付いているはずです。この数字が「変更されたファイルの数」です。クリックしてみましょう。

 スクショ：ForkのLocal Changes画面

5. 画面が上下2つに分かれています。ここがコミットの操作盤です。
 - 上（Unstaged）：変更されたけど、まだセーブ対象に選んでいないファイル
 - 下（Staged）：セーブ対象に選んだファイル

用語：ステージング

「この変更をセーブに含める」と選ぶ作業。カメラのファインダーを覗いて、写す範囲を決める作業だと思ってください。全部写す必要はなく、今回の作業に関係するものだけ選べます。

6. ファイルを選んで「Stage」ボタン（または右クリック→Stage）を押すと、下に移動します。
全部まとめて送るなら「Stage All」でOK。
※このとき、右側にそのファイルの変更内容が表示されます。画像なら**変更前と変更後を並べて見られる**ので、デザイナーさんには嬉しい機能です。
7. 【1-05の復習】.metaファイルも一緒にステージしてください。
画像を1枚追加すると「Player.png」と「Player.png.meta」の2つが出てくるはずです。必ず**セット**で。片方だけコミットすると、相手の環境で絵が外れます。
8. 下の入力欄に、コミットメッセージを書きます。ここが実は一番大事です。

用語：コミットメッセージ

「この変更で何をしたか」の一言メモ。未来の自分とチームメンバーへの手紙です。

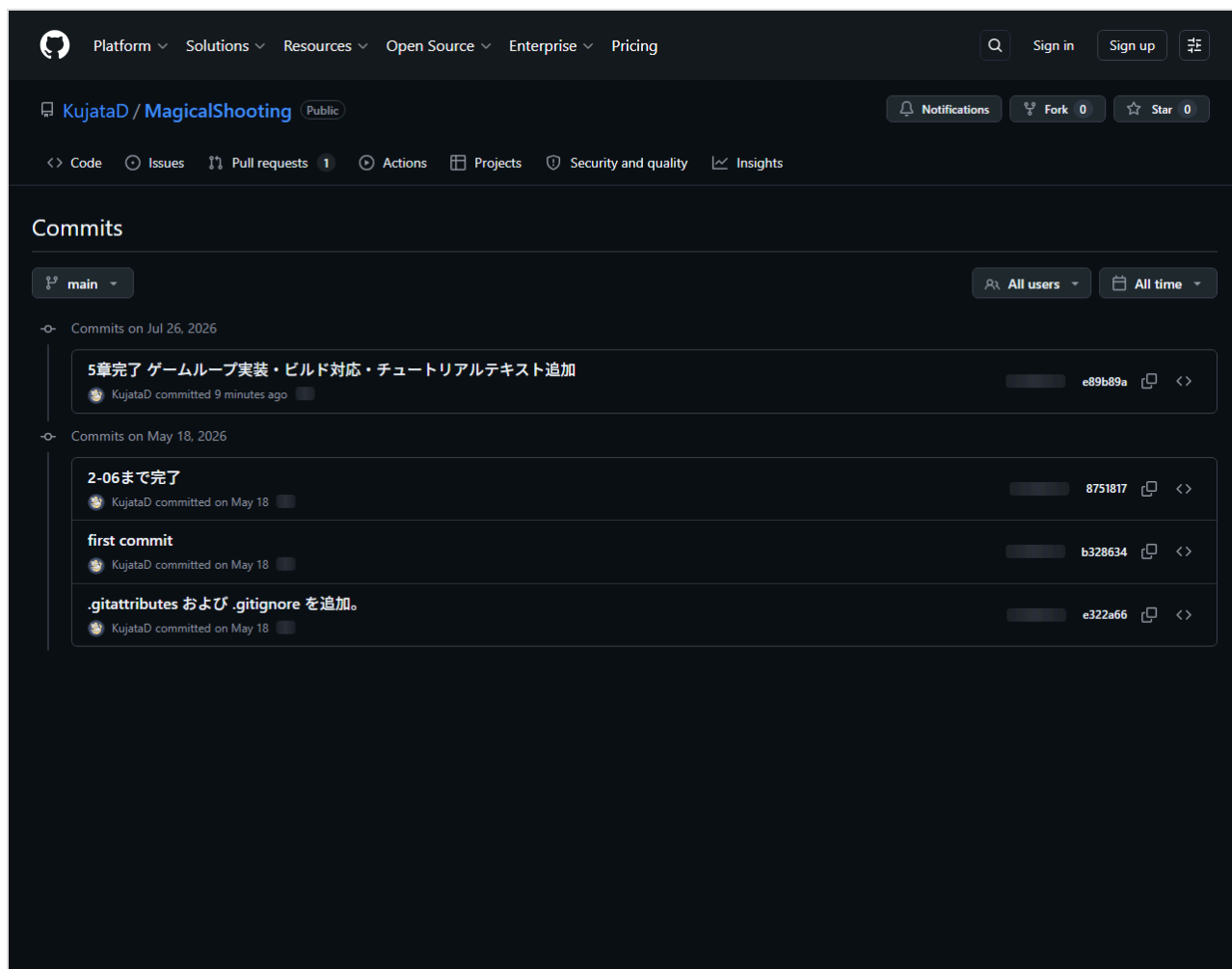
9. 良いメッセージと悪いメッセージの例を挙げます。

✖ 悪い例	○ 良い例
修正	敵キャラの当たり判定を小さく修正
aaa	タイトル画面のロゴ画像を差し替え
いろいろ	プレイヤーの移動速度を5.0に調整
(空欄)	ゲームオーバー時のBGMを追加

コツは「何をどうしたか」を書くこと。3ヶ月後の自分は、絶対に「修正」の中身を覚えていません。

10. メッセージを書いたら「Commit」ボタンを押します。これでセーブ完了！

11. 左メニューの「All Commits」（履歴）を見てみましょう。あなたのコミットが、名前とメッセージ付きで一番上に追加されているはずです。



12. 【コツ】コミットは細かく、こまめに

「1日の終わりにまとめて1回」ではなく、「1つの作業が終わるたびに1回」がおすすめです。

- ✖ 「今日やったこと全部」（何かあった時に戻る場所が粗い）

-  「敵の絵を追加」「敵のサイズ調整」「敵の色味修正」（ピンポイントで戻れる）

セーブポイントは、多くて困ることはありません。

13. コミットが履歴に表示されたら、今回は完成です！

.....ただし、これはまだ**自分のPCの中**だけの話。チームには何も届いていません。次回、いよいよ倉庫に送ります。

14. ↓ --- **質問コーナー** --- ↓

【1-09】プッシュ&プルしよう（納品と仕入れ）【1章：Git導入編】

1. クリア条件：自分のコミットをGitHubに送り、メンバーの変更を受け取る
2. 前回のコミットは、実はまだ**自分のPCの中だけ**の出来事でした。
Gitは「手元の倉庫」と「ネットの倉庫（GitHub）」の2階建てになっていて、この2つは自動では同期しません。自分のタイミングで送ったり受け取ったりします。
3. 今回覚える操作は2つ。往復するだけです。

操作	向き	たとえるなら
プッシュ (push)	自分 → GitHub	納品する
プル (pull)	GitHub → 自分	仕入れる

用語：プッシュ / プル

push は「押し出す」、pull は「引っぱる」。自分から見て、荷物を押し出すのか、引き寄せるのか、と覚えると迷いません。

4. まずプッシュしてみましょう。Forkの上部ツールバーにある「Push」ボタン（↑の矢印アイコン）を押します。

 スクショ：ForkのPushボタン

5. 確認ダイアログが出るので、そのまま「Push」を押せばOK。
数秒待って、エラーが出なければ納品完了です。
※初回はGitHubのログインを求められることがあります。1-04で連携していれば自動で通ります。
6. GitHubのリポジトリページをブラウザで開いて、リロードしてみてください。
さっきのコミットメッセージが、ちゃんとサイトに反映されているはずですよ。これでチーム全員があなたの作業を見られる状態になりました。
7. 次はプル。他のメンバーが納品したものを受け取る操作です。
Forkの「Pull」ボタン（↓の矢印アイコン）を押すだけ。

 スクショ：ForkのPullボタン

8. 【超重要】作業を始める前は、必ずプルする
これはチーム制作の**鉄の掟**です。理由を絵で説明します。

× 悪い例：プルしないで作業
Aさん：古いデータのまま作業 → 納品しようとする
GitHub：「その間に他の人が更新してるよ」→ 拒否される
Aさん：手作業で合体させるハメに（つらい）

○ 良い例：作業前にプル

Aさん：最新を取得 → 作業 → 納品（すんなり通る）

Unityを開く前に、まずForkでPull。これを習慣にしてください。朝イチのメールチェックと同じだと思ってください。

9. 【Unity特有の注意】プルする前に、Unityは閉じておく

Unityを開いたままプルすると、Unityが「開いているファイルが勝手に書き換わった！」と混乱して、最悪プロジェクトが壊れます。

安全な手順はこれです。

10. Unityで作業 → 保存 → **Unityを閉じる**

11. Forkでプル

12. Unityを開き直す

※慣れてくると開いたままでも大丈夫な場面は分かりますが、最初は必ず閉じましょう。

13. 【エラーが出たら】「Push rejected」と言われた場合

これは「あなたが作業している間に、他の人が先に納品したよ」という意味です。焦らなくて大丈夫。

14. まず **Pull** する（相手の変更を受け取る）

15. 自動で合体できれば、そのまま **Push** し直す

16. 合体に失敗した（コンフリクト）場合は、1-13で対処法をやり直す

17. プッシュしてGitHub上で確認できて、プルもできたら、今回は完成です！

これでGitの基本操作（コミット・プッシュ・プル）は全部そろいました。次回からは、チームで衝突しないための仕組みに入ります。

18. ↓ --- **質問コーナー** --- ↓

【1-10】ブランチってなに？【1章：Git導入編】

1. クリア条件：ブランチを1本作って、切り替えられるようになる
2. ここまでで、1人での作業はできるようになりました。でもチーム3人が同じ場所に納品し続けると、こうなります。
 - Aさんが作業中の「まだ動かない敵キャラ」を納品
 - Bさんがそれを受け取る → ゲームが起動しなくなる
 - Bさん「動かないんだけど！」
 - Aさん「ごめん、作業途中で……」

よくある事故です。これを防ぐのがブランチです。

3. ブランチとは、作業用のパラレルワールドです。

用語：ブランチ（branch）

直訳は「枝」。本流から枝分かれさせた、独立した作業スペースのこと。枝の中で何をしても本流には影響しません。作業が完成してから、枝を本流に合流させます。

4. デザイナーさん向けのたとえで言うと、**レイヤーの複製**が近いです。
 - 元のレイヤー（main）はそのまま残す
 - 複製したレイヤー（feature）で好きなだけ試す
 - うまくいったら統合する。ダメなら複製ごと捨てる

Photoshopで直接背景レイヤーを塗りつぶす人はいませんよね。それと同じ話です。

5. もうひとつのたとえはノートです。
 - **main** = 清書ノート（きれいな状態を保つ。人に見せられる）
 - **feature** = 下書きノート（ぐちゃぐちゃに書いてOK。失敗し放題）

6. では実際に作ってみましょう。Forkの左メニューに「Branches」という項目があり、今は **main** だけがあるはずです。

7. ブランチを作るには、Repository > New Branch（またはツールバーの「New Branch」ボタン）を押します。

 スクショ：Forkのブランチ作成画面

8. 名前を付けます。ここでルールがあります。

feature/作業内容

例：

- `feature/enemy-graphics`（敵の絵を作る）
- `feature/title-ui`（タイトル画面のUI）
- `feature/bgm`（BGM追加）

「/」で区切ると、Forkの画面上でフォルダのようにまとまって見やすくなります。日本語も使えますが、トラブルの元なので**半角英数**推奨です。

9. 「Create Branch」を押すと、新しいブランチができて、自動でそちらに切り替わります。

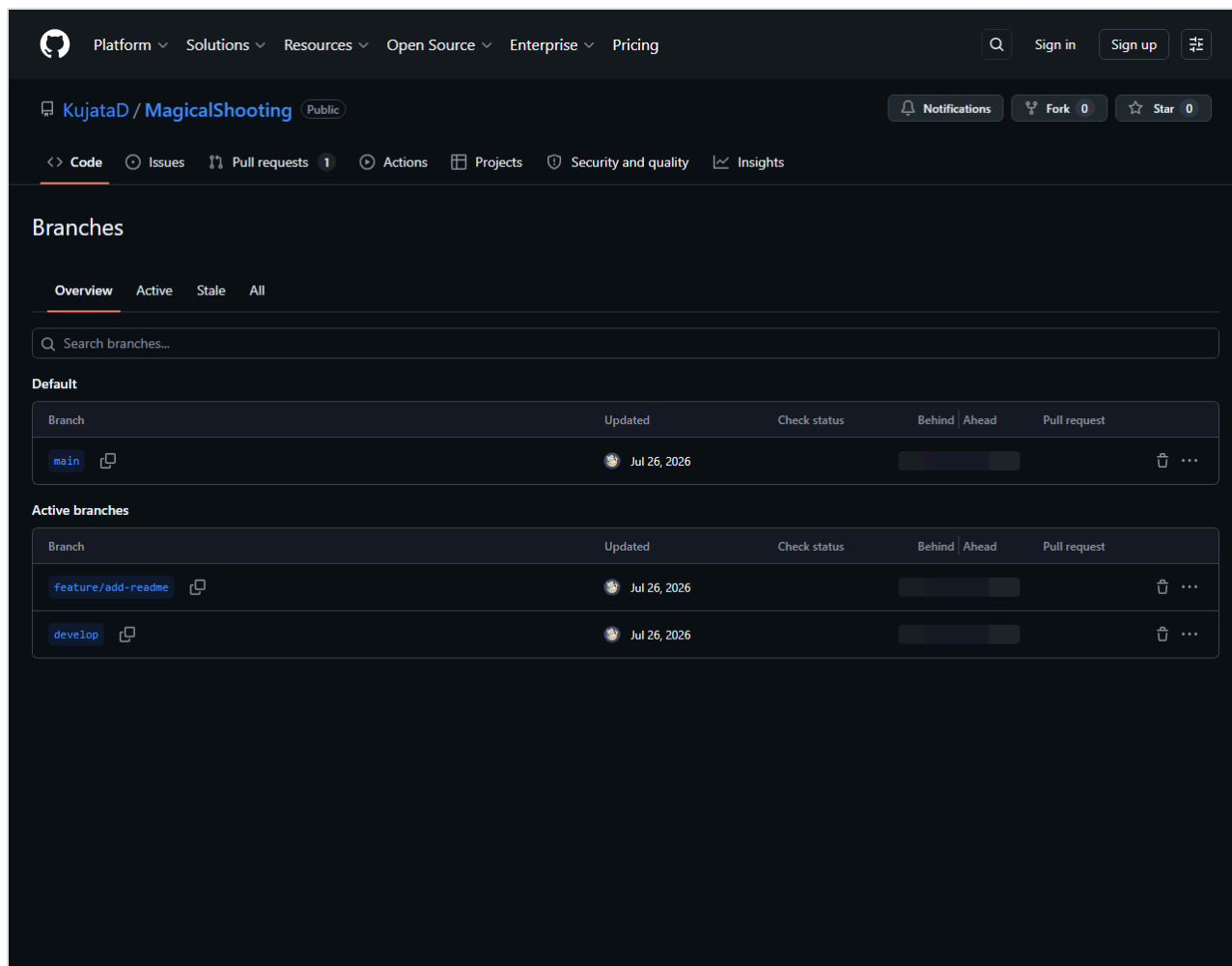
用語：チェックアウト（checkout）

ブランチを切り替えること。切り替えると、フォルダの中身が一瞬でそのブランチの状態に入れ替わります。魔法みたいですが、Gitが履歴から復元しているだけです。

10. 【重要】今どのブランチにいるか、必ず確認するクセをつける

Forkの左メニューで、**太字になっているのが現在のブランチ**です。作業を始める前にここをチラ見してください。

「mainで作業しちゃった！」は初心者あるあるのミスNo.1です（対処法は1-14の逆引き表に載せます）。



11. 試しに、ブランチを行ったり来たりしてみましょう。

- featureブランチで何かファイルを追加してコミット
- mainにチェックアウト（ダブルクリックで切り替え）
- エクスプローラーでフォルダを見ると、追加したファイルが消えている！
- featureに戻すと、また現れる

消えたわけではなく、パラレルワールドを移動しているだけです。これがブランチの感覚です。

※Unityを開いたままブランチを切り替えると混乱するので、切り替え時もUnityは閉じておきましょう。

12. ブランチを作って、切り替えができれば、今回は完成です！

次回は、チームでどうブランチを使い分けるかのルールを決めます。

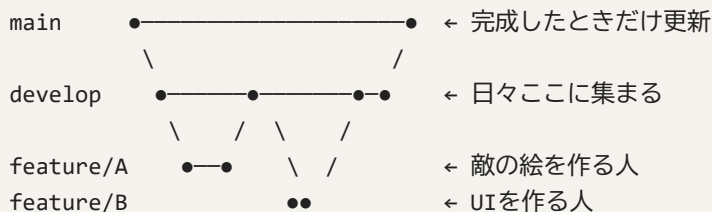
13. ↓ --- **質問コーナー** --- ↓

【1-11】main / develop / feature で運用しよう【1章：Git導入編】

1. クリア条件：3種類のブランチの役割を理解して、チームのルールを決める
2. 前回ブランチを作れるようになりました。でも全員が好き勝手にブランチを作ると、それはそれで混乱します。
そこで、チームで**3層構造**のルールを決めます。これは世界中のチームが使っている定番のやり方です。
3. 3つのブランチの役割はこうです。

ブランチ	役割	たとえるなら
main	完成版。いつでも遊べる状態を保つ	本番放送 / 提出用の清書
develop	開発版。みんなの作業がここに集まる	リハーサルスタジオ
feature/〇〇	個人の作業用。1タスク1本	各自の個人練習室

4. 図にするとこうです。作業は下から上に合流していきます。



5. ルールはシンプルに3つだけ。
 - **main**には**直接コミットしない**（絶対）
 - **develop**にも**直接コミットしない**（原則。PR経由で入れる）
 - **作業は必ずfeatureブランチを切ってから**

「じゃあどこで作業するの？」→ **常にfeatureです**。これだけ覚えてください。

6. 【なぜmainを守るのか】

締切当日に「とりあえず今の状態を提出しよう」となったとき、**main**がいつでも動く状態になっていれば、そこから出せば済みます。

逆に**main**が壊れていると、締切直前に地獄を見ます。**mainは避難場所**だと思ってください。

7. **featureブランチの名前は、作業内容がわかるように付けます。**

feature/enemy-graphics	敵キャラの絵
feature/title-ui	タイトル画面

feature/bgm-stage1	ステージ1のBGM
feature/fix-player-speed	プレイヤー速度の調整

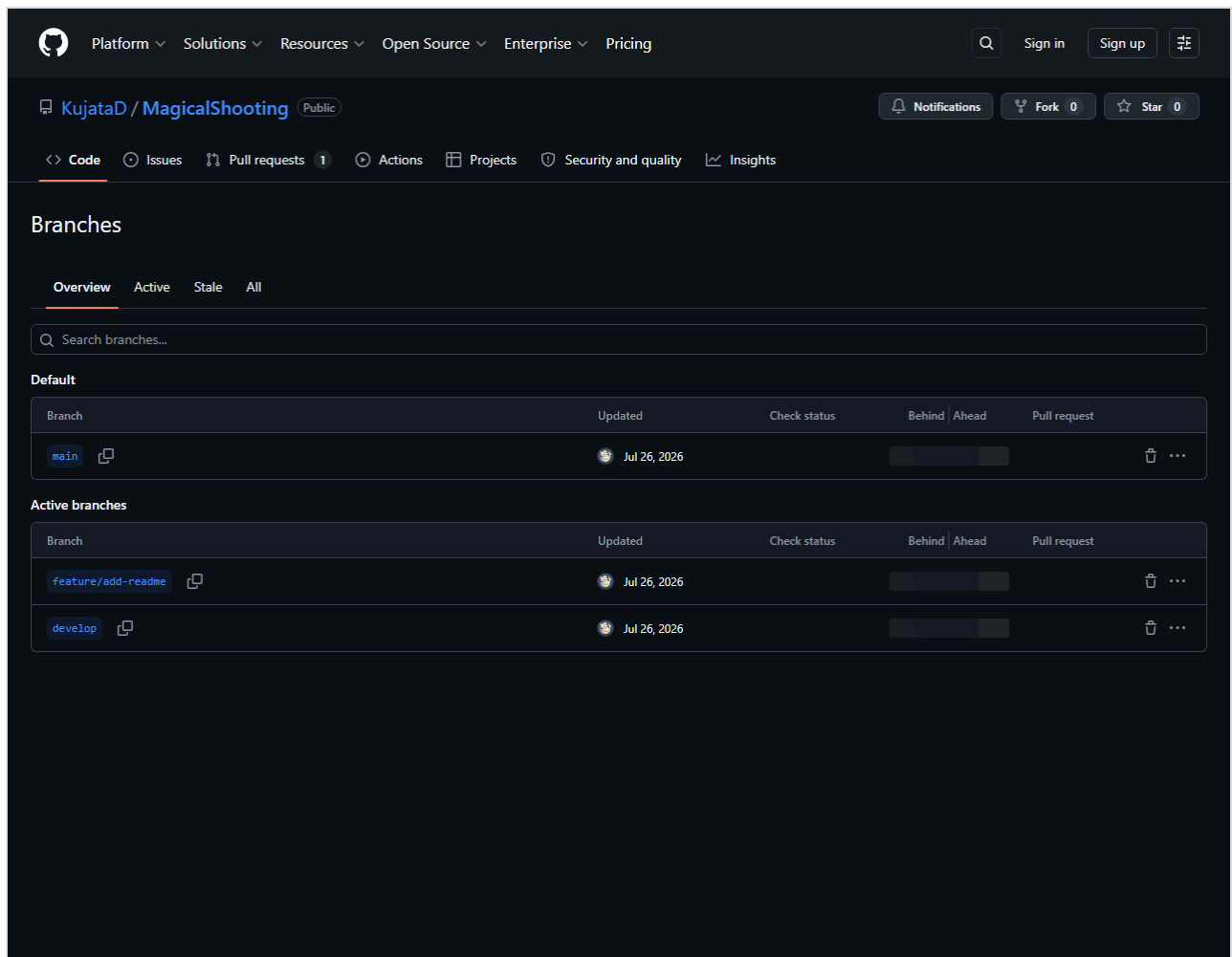
ポイントは「1つのブランチ=1つのタスク」。

「今日やること全部」を1本のブランチに詰め込むと、レビューする人が泣きます。だいたい半日～2日で終わる粒度が目安です。

8. では実際に、チームのdevelopブランチを作りましょう（リーダーが1回だけやればOK）。

- Forkで `main` にいることを確認
- New Branch → 名前を `develop` にして作成
- **Push** して、GitHubにもdevelopを作る

9. GitHubのリポジトリページで「Branches」を見ると、mainとdevelopの2本があるはずです。



10. 【リーダー向け・強くおすすめ】mainを保護する

GitHubには「このブランチには直接コミットさせない」という設定があります。事故を仕組みで防げるので、設定しておく心安心です。

- リポジトリの `Settings > Branches`

- 「Add branch ruleset」または「Add rule」
- Branch name pattern に `main` を入力
- 「Require a pull request before merging」にチェック

これで、うっかりmainに直接プッシュしようとしても、GitHubが止めてくれます。

11. 【メンバー全員でやること】自分の作業ブランチを作る

developができたら、各自こうします。

12. Forkで `develop` にチェックアウト

13. Pull（最新のdevelopにする）

14. New Branch で `feature/自分のタスク名` を作成

15. そこで作業開始！

※featureは**必ずdevelopから枝分かれさせる**のがポイントです。mainから作ると、他の人の作業が入っていない古い状態から始まってしまいます。

16. developブランチができて、各自がfeatureブランチを持てたら、今回は完成です！

次回は、featureで終わった作業をdevelopに合流させる手続きをやります。

17. ↓ --- **質問コーナー** --- ↓

【1-12】プルリクエストを出そう【1章：Git導入編】

1. クリア条件：featureブランチからdevelopへのプルリクエストを1本作る
2. featureブランチで作業が終わりました。これをdevelopに合流させたいのですが、勝手に合流させると誰も気づきません。そこで、「合流させていいですか？」とチームにお伺いを立てる手続きを踏みます。これがプルリクエストです。

用語：プルリクエスト（Pull Request、略してPR）

「私の作業を取り込んでください」というお願い。**提出箱**、あるいは**校正依頼**だと思ってください。出すと、他のメンバーが中身を確認して、OKなら合流させます。

3. デザインの現場で言えば、**クライアントチェック**に近いです。

- 自分の作業（feature）が完成した
- いきなり本番に載せず、まず確認してもらおう（PR）
- 修正指摘があれば直す
- OKが出たら本番に反映（マージ）

4. まず、作業をプッシュします。1-09でやったやつです。

Forkで自分のfeatureブランチにいることを確認して、コミット → **Push**。

※初回のプッシュ時は「Publish branch」のような表示になります。これはGitHub側にまだこのブランチが無いので、新しく作るという意味です。そのままPushでOK。

5. プッシュしたら、GitHubのリポジトリページをブラウザで開いてください。

黄色い帯で「feature/○○ had recent pushes」と表示され、「Compare & pull request」という緑のボタンが出ているはずです。これを押します。

※出ていない場合は、上部メニューの「Pull requests」→「New pull request」からでもOK。

6. 【最重要】合流先（base）の確認

PR作成画面の上部に、こういう表示があります。

```
base: main ← compare: feature/enemy-graphics
```

ここのbaseを **develop** に変更してください。

デフォルトだとmainになっていることが多いですが、1-11のルール通り、featureはdevelopに合流させます。

```
base: develop ← compare: feature/enemy-graphics
```

○ これが正解

7. タイトルと説明を書きます。

- **タイトル**：何をしたか一言（例：「敵キャラのグラフィックを追加」）
- **説明**：もう少し詳しく

説明のテンプレートを置いておきます。これに沿って書けば十分です。

```
## やったこと
- 敵キャラ（お化け）の画像を3種類追加
- Enemyプレハブの見た目を差し替え

## 確認してほしいこと
- サイズ感が他のキャラと合っているか
- 色味が背景と喧嘩していないか

## スクショ
（ここに画像をドラッグ&ドロップで貼れます）
```

※GitHubの説明欄は、**画像をドラッグ&ドロップでそのまま貼れます**。デザイナーさんは変更前後のスクショを貼ると、レビューする人がとても助かります。

8. 「Create pull request」を押せば、PRの完成です！

The screenshot shows a GitHub Pull Request (PR) titled "READMEを追加 #1" in the repository "KujataD / MagicalShooting". The PR is open, showing a commit from "feature/add-readme" to "develop". The description includes a list of changes and a checklist. A bot review is visible, and the right sidebar shows no assignees, labels, or projects.

READMEを追加 #1

やったこと

- README.md を新規作成（遊び方・開発環境・ブランチ運用表）

確認してほしいこと

- 文言がおかしくないか
- ブランチ運用の表がチームのルールと合っているか

chatgpt-codex-connector [bot] reviewed 3 minutes ago

Codex Review

Here are some automated review suggestions for this pull request.

Reviewed commit: 747a078c76

1 participant

9. PR画面には便利なタブがあります。

- **Conversation** : コメントのやり取り。ここで議論します
- **Commits** : このPRに含まれるコミット一覧
- **Files changed** : **変更されたファイルの中身が全部見られるタブ**。画像も変更前後を並べて表示できます

「Files changed」は、レビューするときのメイン画面です。行にマウスを乗せると「+」が出て、**ピンポイントでコメントを付けられます**。

10. PRを出したら、チームに一声かけましょう。

> 「敵の絵のPR出しました！レビューお願いします 🙏」

GitHubは通知を出しますが、埋もれがちです。DiscordやSlackで言うのが確実です。

11. 【レビューする側になったら】

難しく考えなくて大丈夫です。見るのはこの3点だけ。

- 意図した変更だけが入っているか（関係ないファイルが混ざっていないか）
- 明らかにおかしいところがないか
- 自分の作業と衝突しそうな部分がないか

問題なければ「Files changed」→「Review changes」→「Approve」でOKサインを出します。

気になる点があれば、コメントを残して「Request changes」。遠慮せず質問して大丈夫です。**PRは指摘の場ではなく、共有の場**です。

12. PRが作れて、レビュー依頼まで出せたら、今回は完成です！

次回、いよいよ合流させます。

13. ↓ --- **質問コーナー** --- ↓

【1-13】マージとコンフリクト【1章：Git導入編】

1. クリア条件：PRをマージして、コンフリクトが起きたときの直し方を知る
2. レビューでOKが出ました。いよいよ合流させます。この合流作業がマージです。
3. GitHubのPR画面を下にスクロールすると、緑色の「Merge pull request」ボタンがあります。これを押して、確認の「Confirm merge」を押すだけ。
※ボタンが緑なら「自動で合体できます」という意味。灰色や赤の場合は、後述のコンフリクトが起きています。
4. マージが終わると、「Delete branch」というボタンが出ます。押して大丈夫です。

用語：ブランチの削除

役目を終えたfeatureブランチを消すこと。**中身が消えるわけではありません**。作業内容はdevelopに合流済みで、履歴にも残っています。消すのは「もう使わない枝」だけ。放置するとブランチ一覧がゴミだらけになるので、こまめに消しましょう。

5. マージ後、**チーム全員がやる**ことがあります。
 - > developをプルして、最新にする誰かのPRがマージされたら、他のメンバーのdevelopは古くなります。Discordで「マージしました！」と流して、全員がプルする。これが1サイクルです。
6. ここからが本題。コンフリクトの話です。

用語：コンフリクト（conflict）

直訳「衝突」。2人が同じファイルの同じ場所を別々に書き換えたときに起きる、Gitが自動で合体できない状態のこと。

7. なぜ起きるのか。Gitはこういう判断をしています。

- 自動で合体できる
 - Aさん：3行目を修正
 - Bさん：50行目を修正
 - 場所が違うので、両方採用すればいい
- × 合体できない（コンフリクト）
 - Aさん：3行目を「赤」に変更
 - Bさん：3行目を「青」に変更
 - どちらが正しいの？ Gitには判断できない

Gitは賢いですが、**あなたの意図までは分かりません**。だから「人間が決めてください」と手を止めるわけです。これは故障ではなく、正常な動作です。

8. 【予防が9割】そもそも起こさない工夫

コンフリクトは、直すより起こさない方が100倍ラクです。1-05でも触れましたが、大事なので再掲します。

- 同じシーン・同じプレハブを2人で同時に触らない（Unityで最も事故る）
- 作業前にDiscordで「今日はTitleScene触ります」と宣言する
- featureブランチは小さく、短期間で終わらせる（長く放置するほど衝突する）
- 作業開始前に必ずdevelopをプルする

9. 【起きてしまったら】GitHub上で解決する（簡単なケース）

PR画面に「This branch has conflicts that must be resolved」と出たら、「Resolve conflicts」ボタンを押します。エディタが開いて、こういう記号が表示されます。

```
<<<<<< feature/enemy-graphics
speed = 8.0f;          ← あなたの変更
=====
speed = 5.0f;          ← 相手の変更
>>>>>> develop
```

やることは単純です。

10. 正しい方を残して、いらない方を消す（両方活かす場合は両方残す）

11. <<<<<<、=====、>>>>>> の記号3行も全部消す

12. 「Mark as resolved」→「Commit merge」

※記号の消し忘れが一番多いミスです。消し忘れると、その記号がそのままコードや設定に残って壊れます。

13. 【Unityのシーンでコンフリクトしたら】無理せず作り直す

正直に言います。Unityのシーンファイルのコンフリクトは、手作業で直すものではありません。

中身は数千行の複雑なテキストで、人間が正しく解決するのはほぼ不可能です。

現実的な対処はこれ。

- どちらか一方の変更を丸ごと採用する（相手と話して決める）
- 捨てた側の作業は、Unity上で手作業でやり直す（悔しいですが、これが一番速くて確実）

Forkなら、コンフリクトしたファイルを右クリック → 「Use 'ours'」（自分の方を採用）または「Use 'theirs'」（相手の方を採用）で一発です。

※だからこそ、手順8の予防がすべてなんです。「同じシーンを触らない」は本当に大事。

14. 【最後の手段】どうしようもなくなったら

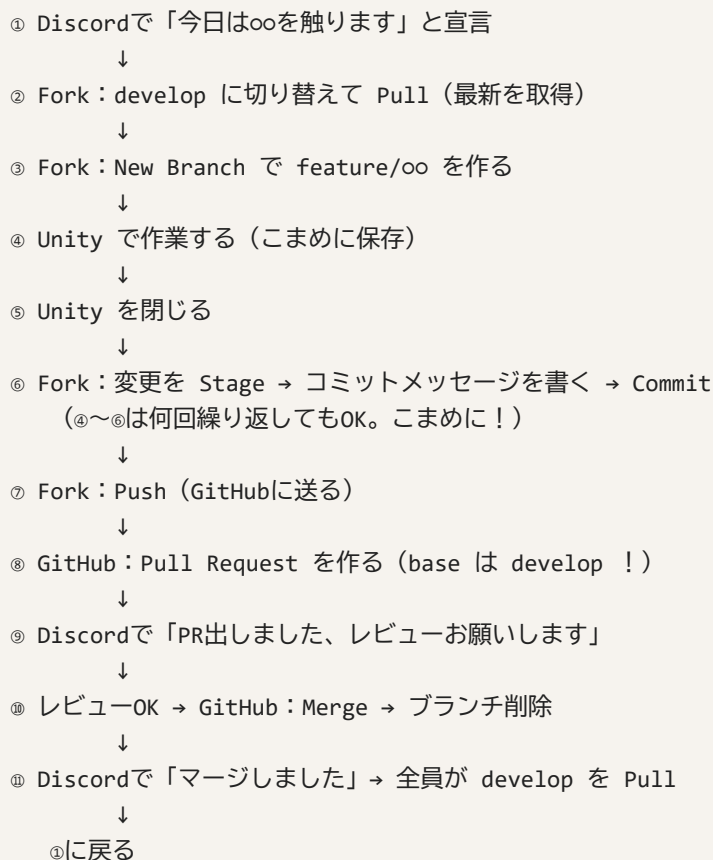
Gitは怖くありません。最悪の場合でも、GitHubに上がっているものは無事です。手元がぐちゃぐちゃになったら、こうすればリセットできます。

15. 今のプロジェクトフォルダを別名にリネーム（念のため取っておく）
16. GitHubからもう一度クローンし直す（1-07の手順）
17. 必要なファイルだけ、古いフォルダから手作業でコピーする
「壊したかも」と思っても、プッシュ済みのものは絶対に消えません。安心してください。
18. PRをマージできて、コンフリクトの直し方が分かったら、今回は完成です！
次回はいよいよ最終回、全部忘れても大丈夫な保存版マニュアルを作ります。
19. ↓ --- **質問コーナー** --- ↓

【1-14】通し：チーム作業分担マニュアル【保存版】

1. クリア条件：これ1枚を見れば、全部忘れていても作業分担ができる
2. 最終回です。ここまでの内容を全部忘れても、このページさえ開けば作業が回るようにまとめました。
印刷するなりDiscordにピン留めするなりして、手元に置いてください。

【A】毎日の作業フロー（これだけ覚えればOK）



【B】ブランチの役割（3つだけ）

ブランチ	役割	触り方
main	完成版。いつでも遊べる状態	直接触らない。リリース時のみ
develop	開発版。みんなの作業の集合場所	直接コミットしない。PR経由で入れる
feature/〇〇	個人の作業場所	ここで作業する。1タスク1本

ブランチ名の例： `feature/enemy-graphics` `feature/title-ui` `feature/bgm-stage1`

【C】Unityチーム制作の鉄の掟（5ヶ条）

1. 同じシーン・同じプレハブを2人で同時に触らない
→ いちばん事故る。作業前にDiscordで宣言する
2. .metaファイルも必ず一緒にコミットする
→ 忘れると相手の環境で絵やスクリプトの紐付けが外れる
3. Pull / ブランチ切り替えのときは Unity を閉じる
→ 開いたままやるとUnityが混乱する
4. Libraryフォルダは共有しない
→ .gitignoreで除外済み。各自のPCで自動生成される
5. 作業前は必ず develop を Pull する
→ 古い状態から作業を始めると、後で合体できなくなる

【D】チームでの分担の決め方

作業がぶつからないように、ファイル単位で縄張りを決めるのがコツです。

良い分担 ○	悪い分担 ×
Aさん：Enemyプレハブ周り Bさん：Playerプレハブ周り	Aさん：敵の動き Bさん：敵の見た目 → 同じファイルに触る
Aさん：TitleScene Bさん：GameScene	2人で同じGameSceneを編集
絵の差し替えはAさん一括	全員が好きなタイミングで素材追加

大きなシーンは、機能ごとにプレハブに分けておくと分担しやすくなります。

（例：UI一式をプレハブ化 → UI担当はプレハブだけ触ればよく、シーン本体は触らない）

【E】困ったとき逆引き表

こうなった	こうする
mainで作業しちゃった！（まだコミット前）	Forkでブランチを作って切り替える。変更はそのまま新ブランチに付いてくる

mainにコミットしちゃった！（まだPush前）	該当コミットを右クリック → 「Reset ... to Here」の前に、まずFork右上でブランチを作成 → その後mainを1つ戻す。不安ならリーダーに相談
Pushが拒否された（Push rejected）	先にPullする → 自動合体できればもう一度Push
コンフリクトした	1-13参照。シーンファイルなら、片方を丸ごと採用してもう片方は手作業でやり直すのが速い
相手の環境だけ絵が消える	.metaファイルのコミット漏れ。Assetsの.metaを全部コミットし直す
プロジェクトが壊れた気がする	フォルダをリネームして、GitHubからクローンし直す。Push済みのものは絶対に消えない
間違えてPRをマージした	GitHubのPR画面に「Revert」ボタンがある。取り消し用のPRが自動で作られる
どのブランチにいるか分からない	Forkの左メニューで太字になっているのが現在地
Unityが「シーンが壊れています」と言う	Unityを閉じて、Forkで該当ファイルを「Discard changes」→ Pullし直す

【F】コミットメッセージの書き方

- 敵キャラのグラフィックを3種類追加
- タイトルロゴのサイズを調整
- プレイヤーの移動速度を5.0に変更

- × 修正
- × いろいろ
- × aaa

「何を」「どうしたか」。3ヶ月後の自分に向けて書きましょう。

【G】1日の終わりチェックリスト

- [] 作業中のファイルは全部コミットした？
- [] featureブランチにPushした？（PCが壊れてもGitHubにあれば無事）
- [] 完成したタスクはPRを出した？
- [] 明日触るファイルをDiscordで宣言した？

3. お疲れさまでした！これでGit編は全部完了です。

最初は用語だらけで大変だったと思いますが、実際に毎日使うのは

Pull → ブランチ作る → 作業 → Commit → Push → PR

この流れだけです。1週間も回せば体が覚えます。

4. そして何より、Gitを覚えると思いついた**試行錯誤**ができるようになります。

「壊したらどうしよう」と思わなくていいので、ブランチを切って好きなだけ実験できる。気に入らなければ捨てればいい。

これはデザイン作業と本当に相性がいいはずです。良いチーム制作を！

5. ↓ --- **質問コーナー** --- ↓